

36. What is Recursion in Python?

Answer:

Recursion is a programming technique where a function calls itself to solve a problem. Every recursive function must have a **base case** to stop the recursion; otherwise, it will result in a RecursionError.

Example:

```
def factorial(n):  
    if n == 1:    # Base case  
        return 1  
    return n * factorial(n - 1)
```

```
print(factorial(5))
```

Output:

120

Interview Tip:

Recursion is commonly used for problems like **factorial**, **Fibonacci series**, and **tree traversal**.

37. What is the difference between Recursion and Iteration?

Answer:

Recursion	Iteration
Function calls itself	Uses loops (for or while)
Requires a base case	Requires a loop condition
Uses more memory (call stack)	Uses less memory
Can be easier to understand for some problems	Usually faster

Interview Tip:

Use **iteration** for better performance and **recursion** when it makes the solution simpler.

38. What is a Lambda Function?

Answer:

A **lambda function** is an anonymous (unnamed) function used for short, simple operations. It is written in a single line using the lambda keyword.

Syntax:

lambda arguments: expression

Example:

```
square = lambda x: x * x  
print(square(5))
```

Output:

25

Interview Tip:

Lambda functions are often used with functions like map(), filter(), and sorted().

39. What are map(), filter(), and reduce()?

Answer:

- **map()** → Applies a function to every item in an iterable.
- **filter()** → Selects items that satisfy a condition.
- **reduce()** → Combines all items into a single value (available in the functools module).

Example:

```
numbers = [1, 2, 3, 4]  
  
print(list(map(lambda x: x * 2, numbers)))  
print(list(filter(lambda x: x % 2 == 0, numbers)))
```

Output:

```
[2, 4, 6, 8]  
[2, 4]
```

Interview Tip:

Know the purpose of each function:

- `map()` → Transform data
- `filter()` → Select data
- `reduce()` → Aggregate data

40. What is the difference between `append()` and `insert()`?

Answer:

`append()`

Adds an element at the end of the list

Takes one argument (the value)

`insert()`

Adds an element at a specific index

Takes two arguments (index, value)

Example:

```
numbers = [1, 2, 3]
```

```
numbers.append(4)
```

```
print(numbers)
```

```
numbers.insert(1, 10)
```

```
print(numbers)
```

Output:

```
[1, 2, 3, 4]
```

```
[1, 10, 2, 3, 4]
```

Interview Tip:

- Use **`append()`** when adding an element to the end of a list.
- Use **`insert()`** when placing an element at a specific position.